

VoiceDeskElectron — architecture plan

Living document. Every sentence here is true of the current code, or it gets corrected. History lives in git, not in this file.

Repository: VoiceDesktopElectron · **Product name:** VoiceDeskElectron · **Platform:** macOS (Apple Silicon and Intel) · **Canonical version:** the `version` field of `package.json`.

1. What this is

A desktop window with one control. You **hold** a button (or the `Space` key while the window has focus), speak, and release. Your words appear as text. The text is handed to a **coding agent running through its own CLI** — not the model API — which reads and writes Markdown files in a `notes/` folder beside the app. The agent's reply appears in the window, and can be spoken back.

Two sentences define the whole product:

- *"add milk to my shopping list"* → the agent creates or edits `notes/shopping.md`.
- *"what's on my list?"* → the agent answers from that file.

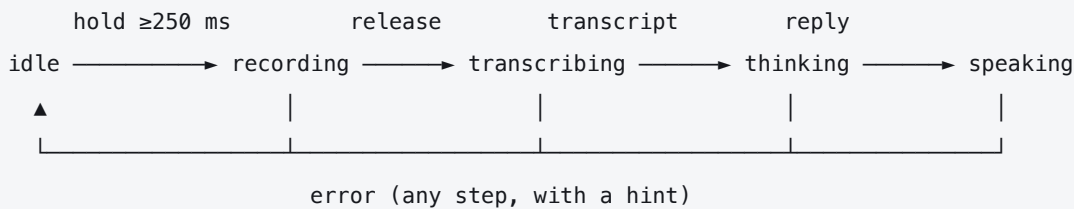
Non-goals

Named so they are not rediscovered as missing features:

- **Not a chat app.** One turn at a time. No conversation history UI beyond the current turn and a short scrollbar.
 - **Not a notes editor.** The app never writes `notes/` itself. Every byte in there is written by the agent, because "the agent can read and write files" is the thing being demonstrated.
 - **Not multi-agent, not multi-window, not cross-platform.** One window, one agent, macOS.
 - **Not a background dictation daemon.** Push-to-talk works while the window has focus. §6 records what that costs and what the alternative would have cost.
-

2. The turn

The entire product is one flow, and it is a state machine. Every rule below hangs off it.



```
// src/domain/model/turn.ts
export type TurnState =
  | { k: 'idle' }
  | { k: 'recording'; startedAt: number; level: number }
  | { k: 'transcribing' }
  | { k: 'thinking' }
  | { k: 'speaking' }
  | { k: 'error'; failure: TurnFailure }
```

One tagged union, not four booleans. `isRecording && !isTranscribing` invites a state that must not exist; the union makes "recording while thinking" unrepresentable and gives the UI a single value to render.

Edge rules that belong to the machine, not to the UI:

- a hold shorter than **250 ms** is discarded — an accidental tap otherwise produces an empty transcript and a confused agent turn;
- key **repeat** is ignored, and key-down is ignored unless the state is `idle`;
- `pointerup`, `pointerleave`, `pointercancel` and window `blur` **all** end the hold, and the button uses `setPointerCapture` — a recording that never stops is the bug this prevents;
- every turn ends with `stream.getTracks().forEach(t => t.stop())`, or the macOS recording indicator stays lit and the user stops trusting the app.

3. Architecture: the process split *is* the layer boundary

Electron already runs three processes with different privileges. Drawing application layers anywhere else means maintaining two boundaries that will disagree.

process	privilege	is the layer	may import
main	full Node + OS	infrastructure and composition root — "the backend"	anything
preload	bridge only	the contract	electron + shared types
renderer	web page	presentation	shared types, the bridge, and domain/model — pure types and pure functions. Never domain/ports, domain/usecases or infrastructure
domain	<i>not a process</i>	policy: use cases, models, ports	nothing platform-shaped

domain/ is plain TypeScript that would run in a browser, in a test, or in a CLI. It is why the tests need no Electron at all.

That is also why the renderer row above says domain/model rather than "nothing else". It used to say the narrower thing and was contradicted by this very paragraph three lines later, and by the code: useTurn.ts runs nextTurnState and MIN_HOLD_MS as VALUES, because running the turn machine in the renderer is the only way to get the zero-latency key handling §2 requires — a round trip to main to learn whether a keypress starts a recording is a round trip the user feels. The line that matters is not "may the renderer see the domain" but "may it see a SEAM": a port or an adapter in a sandboxed page is Node code where there is no Node.

Both halves are now gated rather than described. test/architecture.test.ts checks that domain/ imports no platform — including the dynamic, side-effect, require and bare-builtin forms its first regex missed, and process.env, which needs no import at all — and, in the other direction, that nothing under src/renderer/ imports infrastructure, domain/ports or domain/usecases.

Folder layout

```

src/
  domain/
    model/turn.ts           TurnState, TurnFailure
    model/transcript.ts     Transcript
    model/agent-reply.ts    AgentReply
    ports/Transcriber.ts    interface Transcriber
    ports/AgentRunner.ts    interface AgentRunner
    ports/SpeechSynthesizer.ts interface SpeechSynthesizer
    usecases/runVoiceTurn.ts audio → transcript → reply  (takes ports as arguments)
  infrastructure/
    transcribe/WhisperCppTranscriber.ts
    transcribe/wav.ts       Float32Array → 16-bit PCM WAV (pure)
    agent/ClaudeCliAgentRunner.ts
    speech/MacSaySynthesizer.ts
    process/resolveBinary.ts PATH-independent binary lookup
  main/
    index.ts                app lifecycle, the single window
    composition-root.ts     the ONLY place an adapter is constructed
    ipc.ts                  handlers: unwrap → use case → wrap
  preload/index.ts          contextBridge surface
  renderer/
    i18n/                  t(), en.json
  shared/ipc.ts             channel names + payload schemas – imported by all three

```

The dependency rule, in one line: arrows point inward. `infrastructure` imports `domain`; the composition root imports both; `domain` imports nothing from the platform.

No `utils/`, no `helpers/`, no root `types/`. A folder with no dependency direction collects everything and the arrows disappear.

The rule is enforced by a test, not by this paragraph

```

// test/architecture.test.ts – shape only; the file is the authority
const FORBIDDEN = [
  { what: 'a static import of the platform',      pattern: /from\s*['"](\s*)['"]/ },
  { what: 'a side-effect import (no `from`)',     pattern: /\s*import\s*['"](\s*)['"]/m },
  { what: 'a dynamic import',                     pattern: /import\s*\(\s*['"](\s*)['']/ },
  { what: 'a require',                            pattern: /require\s*\(\s*['"](\s*)['']/ },
  { what: 'the `process` global, which needs no import at all', pattern: /\bprocess\s*\.\s*(env|plat
]

```

A list of patterns, not one regex, and that is the second version of this gate. The first was a single `/from\s*['"](electron|node:|fs|path|child_process|os)/`, which catches an ordinary `import x from "electron"` and nothing else. This project shows the gate as its flagship example of *a rule that is true*

rather than merely written, and the deliberate violation it was proven on was the easiest possible one. A side-effect `import "electron"`, an `await import("node:...")`, a `require`, any Node builtin outside that four written without the `node:` prefix, and `process.env` — which needs no import, so no import scan of any strictness would ever have found it — all walked straight through.

The gate is proven by breaking it: every pattern is also checked against a sample that *should* trip it, so the suite can tell "the rule holds" from "the rule is a regex that matches nothing" — a typo in one alternation produces the second while looking exactly like the first. The non-empty assertion is there because the classic silent pass is a glob that matched nothing. And the same file now checks the other direction, which nothing checked at all: that nothing under `src/renderer/**` reaches `infrastructure`, `domain/ports` or `domain/usecases`.

Composition root: exactly one

```
// src/main/composition-root.ts
export function buildPorts(env: Env): Ports {
  const transcriber: Transcriber      = new WhisperCppTranscriber(env.whisperBin, env.modelPath)
  const agent: AgentRunner            = new ClaudeCliAgentRunner(env.agentBin, env.notesDir)
  const voice: SpeechSynthesizer      = new MacSaySynthesizer(env.voice)
  return { transcriber, agent, voice, notes: new MarkdownNotesFolder(env.notesDir) }
}
```

Selection happens **by outcome** — platform, env var, availability — in this one file. A `new WhisperCppTranscriber()` anywhere else is a dead seam. IPC handlers are adapters too: unwrap, validate, call the port, wrap. A handler containing business logic has moved policy into the framework.

It returns the ports, not one assembled use case, and that is deliberate. An earlier version of this example returned `{ runVoiceTurn: makeRunVoiceTurn(...) }` and told handlers to "call the use case" — which contradicted §7 three sections later, where the whole reason there are three channels is that the transcript must reach the screen *before* the agent starts thinking. One IPC call cannot answer an interface that needs three returns at three different times, so the handlers drive the ports and the sequencing lives in the renderer's `useTurn`.

`makeRunVoiceTurn` is the same flow written as a pure function, and it is kept: it is how the policy — silence ends the turn, a voice that will not start is not a failed turn — is tested with no Electron, no binaries and no network, which is S3's acceptance criterion. It is a second, documented shape of the same thing, not dead code, and this paragraph is the record that it was a choice.

4. The three seams

Each port is named for its **role**; each adapter for its **technology**. That naming is what tells you which side of the line a file is on.

port (domain)	adapter that ships	second implementation, in the repo on day one
Transcriber	WhisperCppTranscriber	the test double — every domain test needs one, because the real adapter spawns a process
AgentRunner	ClaudeCliAgentRunner , and only <code>claude -p</code>	the test double, same reason
SpeechSynthesizer	MacSaySynthesizer	the null implementation used when speech is off, plus the test double

Only one agent CLI is implemented. `codex exec` and `cursor-agent -p` are future work, not scaffolding to be written now.

That makes the usual objection fair, so it gets answered here rather than in review: *an interface with one implementation is an over-build*. It would be — if the count were one. It is not. Every adapter above reaches the OS, so **every test of the use case needs a substitute**, and that substitute is a real second implementation living in this repository from the first test onward. The port is what makes the domain testable without a microphone, a model file and a spawned agent; the fact that it also makes a second CLI cheap later is a side effect, not the argument.

The litmus stays sharp for everything else: if a future port cannot name its second implementation — a test double counts, "someday" does not — it does not get written.

```
// src/domain/ports/AgentRunner.ts
export interface AgentRunner {
  run(prompt: string, ctx: TurnContext): Promise<AgentReply>
}
```

The use case says *"ask the agent"*. If the string `-p` appears anywhere outside `ClaudeCliAgentRunner` , the seam has leaked.

5. The agent CLI is a process boundary, and gets network-call treatment

The CLI already carries the agent loop, the file-editing tools, permission gating and session state. Calling it means shipping a harness rather than rebuilding a chat client. The cost is a dependency on a program that can hang, die, print garbage, or do more than it was asked.

5.1 `execFile` with an argv array — never a shell

```
execFile(bin, args, { cwd: notesDir, timeout: 90_000, maxBuffer: 8 * 1024 * 1024 }) // 
exec(`${bin} -p "${text}"`) // 
```

A dictated sentence must never reach a shell parser. `"add milk; rm -rf ~"` is one apostrophe away from ordinary speech. An argv array has no quoting to get wrong.

It also removes the login-shell surprise, and on this machine that surprise is **already present**: `claude` resolves to a *zsh function*, not to a binary. An interactive shell sees the function; `execFile` never will — so the flags the agent runs with are exactly the ones §5.2 lists, and nothing an operator's shell profile or wrapper function adds to them. §5.6 is the other consequence.

5.2 The flags, and why each one is there

Every flag is a value chosen here, not a default inherited from someone else. Verified against the CLI actually installed (2.1.263) — the version is recorded in the README, because these are another program's flags and they can move.

flag	why
<code>-p</code>	non-interactive: print the answer and exit. It is a BOOLEAN flag; the prompt is not its argument — see below
<code>--output-format stream-json</code> <code>--verbose</code>	a machine-readable envelope. Text output is for humans and changes without warning. <code>stream-json</code> and not <code>json</code> , because only the streamed events carry the agent's tool calls, and the tool calls are the only way the notes panel can tell a file the agent rewrote from one it merely opened (§7). <code>--verbose</code> is not optional: the CLI refuses the combination without it, at runtime, and its help text does not say so
<code>--model haiku</code>	the cheapest current tier, pinned by alias rather than by a dated snapshot: an alias cannot resolve to an Opus-tier model, it follows the cheap tier when the cheap tier moves, and it outlives the retirement of any one snapshot. <code>VOICEDesk_AGENT_MODEL</code> (default <code>haiku</code>) is the deliberate opt-in for something stronger
<code>--allowedTools</code> <code>Read,Write>Edit,Glob,Grep</code>	exactly the capability the feature needs — read and write notes
<code>--restricted</code>	removes the built-in command-running tools and web fetch entirely. Belt to the allowlist's braces
<code>--add-dir <notesDir></code>	states the writable area explicitly rather than relying on <code>cwd</code> alone
<code>--permission-mode</code> <code>acceptEdits</code>	file edits inside <code>notes/</code> proceed without a prompt. Not <code>bypassPermissions</code> , which would drop the boundary this app is built around
<code>--strict-mcp-config</code>	no MCP servers are inherited from the operator's machine — behaviour that depends on whose config is present is not behaviour that can be tested
<code>--session-id <uuid></code> / <code>--resume <uuid></code>	session continuity, carried explicitly (§5.4)
<code>--append-system-prompt</code> <code><text></code>	tells the agent the notes conventions: one Markdown file per subject, kebab-case names, answer in one short paragraph

Deliberately **not** granted: `Bash` and every other command-running tool. "The agent might need it" trades the entire permission boundary for a maybe.

The prompt travels on stdin, not in the argv. Verified against 2.1.263, and for two reasons.

The first is that it cannot go in the argv safely here. `--add-dir` and `--allowedTools` are **variadic**: they consume every following bare word, so a prompt sitting after either of them is swallowed as one more directory, and the CLI then answers *"Input must be provided either through stdin or as a prompt argument"* — having just eaten the prompt it is asking for. Ordering the argv around that is a rule someone has to keep remembering; stdin removes the class.

The second is that a process argument is world-readable. Everything the user dictates would otherwise sit in `ps` output for every account on the machine, which is a strange property for an app whose entire input is someone talking.

The argv is built by one exported pure function, so the flag set above is **asserted** by `test/claude-cli-agent.test.ts` rather than described here: the permission boundary this app is built around IS this list, and a flag quietly dropped in a refactor breaks nothing that an ordinary test would notice.

Why the model is pinned, and pinned to an alias. This is a demonstration application, and a demo that quietly reaches for an expensive tier bills someone for a decision nobody made. At Anthropic's published list rates Haiku 4.5 is \$1 per million input tokens and \$5 per million output against \$5 / \$25 for Opus 5 — five times cheaper on both halves, on a 200 K context window. Those are list rates and they move, which is the other half of the argument for the alias: `haiku` pins *the cheapest current tier*, not a price. The result is auditable rather than trusted — the JSON envelope carries `modelUsage`, keyed by the model that actually ran (observed here: `claude-haiku-4-5-20251001`, `canonicalModel`: `claude-haiku-4-5`), and `total_cost_usd`, which for a one-line probe was about \$0.01.

5.3 Parse the reply, never cast it

```
const raw = JSON.parse(stdout)
const reply = AgentReplySchema.parse(raw)    // zod, at the seam
```

A field that stops arriving must fail loudly at the boundary, not surface as `undefined` three layers up.

5.4 Session continuity is carried, not hoped for

"Add milk to my list" and "what's on my list?" are two spawns. **The decision: the app generates a UUID per app session, passes `--session-id` on the first turn and `--resume` on every turn after.** A resumed session the CLI has forgotten is a *normal outcome*, not a crash: the adapter retries once as a fresh session and says so in the reply metadata. The alternative — a blank agent re-reading `notes/` every turn — also works, and is what the fallback lands on.

5.5 Authentication is the CLI's own keychain login

Decision: VoiceDeskElectron passes no API key, reads no token, and puts no secret in a process argument or an environment variable. `claude` authenticates itself from the credentials the user's own terminal login left in the macOS **login keychain** — a generic-password item under the service name `Claude Code-credentials` in `~/Library/Keychains/login.keychain-db`, whose account attribute is the macOS user. Observed here: a `claude -p ... --model haiku run` exited `0` with `ANTHROPIC_API_KEY` unset in the environment.

That is a smaller blast radius, not a convenience. A credential the app never holds cannot be logged by it, captured in a crash report, or handed to a child process by mistake — "we mishandled a key" is not among this application's failure modes at all. The entry belongs to Claude Code; VoiceDeskElectron checks only that it is there and never reads it.

The second consequence is a failure class. "Nobody ever signed in on this machine" is a **setup** problem with an exact fix — run `c laude` in a terminal and log in — and the presence of that keychain item is checkable *before* anything is spawned. So the adapter preflights it and reports its own outcome, for the same reason a missing binary gets one: a failure filed under the wrong cause sends the user to debug the wrong program.

5.6 Five distinguishable outcomes, never two

outcome	meaning	what the user is told
exit 0 + parseable stdout	success	the reply
non-zero, or unparseable stdout	the agent failed	trimmed <code>stderr</code> , and that it was the agent
ENOENT	setup failure, not agent failure	" <code>c laude</code> was not found — install it, or set <code>VOICEDESK_AGENT_BIN</code> "
no keychain login (§5.5)	setup failure, not agent failure	"Claude Code is not signed in — run <code>c laude</code> in a terminal and log in"
timeout	the call could not end	the child is killed and the turn fails cleanly

The two **setup** rows are the ones that matter most here. An Electron app launched from Finder inherits a **minimal** `PATH` that does not include `~/ .local/bin` or `/opt/homebrew/bin` , so a binary that works in the terminal is missing in the app. `resolveBinary` therefore checks, in order: an explicit env var, a settings value, then a list of known install locations — and when all fail it reports a *setup* problem with the exact fix. Collapsing that into "the agent failed" sends the user debugging the wrong thing.

The signed-out row is the same mistake wearing different clothes: the binary is present, it runs, and the turn still fails — but nothing about this app is broken and the fix is a one-time login. Both are settled before the prompt is ever sent.

6. Voice in: local Whisper

Decision: `whisper.cpp` (the `whisper-cli` binary), running locally, behind the `Transcriber` port.

Why, against the alternatives the brief allows:

option	verdict
whisper.cpp local	✅ no API key, no account, no network, no per-minute cost; already present on this machine via Homebrew; a reviewer's setup is two <code>brew install</code> lines and one model download
hosted Whisper / Deepgram API	a reviewer must supply their own key before the app does anything. Good quality, worse "runs on a clean machine"
macOS on-device dictation	zero setup and excellent, but reaching <code>SFSpeechRecognizer</code> from Electron needs a native helper binary — an hour of Swift and a build step, for a capability the first row already provides

The audio path

Capture lives in the **renderer**, because that is where the browser media stack is and it hands back the level meter for free.

```
getUserMedia → AudioContext({ sampleRate: 16000 }) → AudioWorklet → Float32Array (mono)
  → ArrayBuffer over IPC → main → wav.ts (16-bit PCM) → temp file → whisper-cli → transcript
```

Why there is no `ffmpeg`, stated correctly. `whisper-cli` 1.9.2 accepts `wav`, `flac`, `mp3` and `ogg`, decodes them through `miniaudio` and **resamples internally** — measured here: a 48 kHz and a 16 kHz WAV of the same utterance produced byte-identical transcripts. What it does *not* accept is **WebM/Opus**, which is exactly what `MediaRecorder` produces in Chromium. So the choice is not "resample or use `ffmpeg`", it is:

path	what it costs
<code>MediaRecorder</code> → <code>WebM/Opus</code> → <code>ffmpeg</code> → WAV	a second binary to resolve, a second <code>ENOENT</code> /setup failure class, a second process spawn per turn, an extra install step in the README, and GPL-3.0-or-later in a dependency set that is otherwise entirely permissive
<code>AudioWorklet</code> → raw PCM → 44-byte WAV header	~35 lines of pure, unit-testable code and no dependency at all

The second row wins on every axis, which is why it is the one being built.

- **16 kHz is requested at the source** as an optimisation — a third of the bytes crossing IPC and less for Whisper to resample — **not as a requirement**. If a device hands back 48 kHz the app writes a 48 kHz WAV and `whisper-cli` deals with it. There is no resampling code here, and no guard is needed for a constraint that does not exist.
- `AnalyserNode` drives a visible level meter. A meter is not decoration — it is the only evidence the user has that the microphone is live, and its absence is the most common reason a voice UI feels broken.
- The stream is acquired on the **first hold**, never at startup. Asking for the microphone before the user has pressed anything reads as spyware and burns the permission prompt.
- Microphone permission has **three** outcomes, and the denied-permanently one names the exact place to fix it: *System Settings* → *Privacy & Security* → *Microphone*.
- `whisper-cli -oj` writes a JSON file whose `transcription[].text` is the transcript, so the adapter parses a declared shape rather than scraping stdout.

Voice out (the brief's stretch goal)

Decided: in scope. `say`, the macOS built-in, behind `SpeechSynthesizer`. Zero dependencies, on-device, one `execFile`, and the reply is spoken back after it is shown. It stays this cheap or it does not ship — a neural voice would be a second adapter, not a rewrite.

7. The IPC contract

The preload bridge is a wire format shared by three separately-compiled build outputs, and a trust boundary: one XSS in the app's own UI is a compromised client.

`shared/ipc.ts` **is the authority; the block below illustrates its shape.** Where the two disagree the file is right and this is stale — which has happened. What keeps the file itself honest is `test/wire-matches-domain.test.ts`, whose mutual-assignability assertions fail to compile when a schema and the domain type it mirrors drift apart.

```

// shared/ipc.ts – the single declaration, imported by main, preload and renderer
export const CH = {
  transcribe: 'turn:transcribe',
  ask: 'turn:ask',
  speak: 'turn:speak',
  appInfo: 'app:info',
  notes: 'notes:list',
  // What macOS says about the microphone. The page cannot find this out for itself: its own
  // Permissions API answers about Chromium's per-origin permission, not about the OS's decision.
  micAccess: 'os:mic-access',
  openSettings: 'os:open-settings',
} as const

// There is deliberately no `state: 'turn:state'`. One existed – schema, a `pushTurnState` in
// main, an `onTurnState` on the bridge – with no caller at either end, and it was deleted rather
// than wired up: $11 commits to ONE state machine held in one hook, and a push from main would
// be a second source of the same state.

export const TranscribeReq = z.object({
  pcm: z.instanceof(ArrayBuffer).refine(b => b.byteLength <= 8 * 1024 * 1024),
  // NOT pinned to 16 kHz: $6 asks the device for it as an optimisation and writes whatever rate
  // it actually gets. A literal here would reject exactly the 48 kHz device $6 promises to support.
  sampleRate: z.number().int().min(8_000).max(192_000),
  heldMs: z.number().nonnegative(),
})
export const NoteFileSchema = z.object({
  name: z.string().min(1).max(255),
  status: z.enum(['edited', 'read', 'unchanged']),
})
export const NotesListRes = z.object({ files: z.array(NoteFileSchema) })
export const AskReq = z.object({ text: z.string().min(1).max(4000), sessionId: z.string().nullable() })
export const AskRes = outcomeOf(z.object({
  reply: z.string(), notes: z.array(NoteFileSchema),
  model: z.string(), sessionId: z.string().nullable(), costUsd: z.number().nullable(),
})))

```

Two of those channels were added for the design's left panel, and the reason is worth stating. The canvas keeps a standing list of `notes/*.md` beside the turn, each row marked `EDITED`, `READING` or `UNCHANGED` — the durable evidence of what the agent did, since the reply scrolls away and the folder does not. The contract could not serve it: nothing enumerated the folder **at rest**, before or between turns, and `AskRes.notes` was a flat `z.array(z.string())`, which can say a file was *involved* and never *how*. `notes:list` answers the first and `NoteFileSchema` the second. `os:open-settings` is the microphone board's action, and it takes **no argument on purpose**: a bridge method that accepted a URL would be `shell.openExternal` with extra steps, so main owns the one URL it will open.

Which side supplies which half also matters: the ROWS come from `readdir` and the STATUSES from the agent's own tool calls. A tool call is evidence of intent; the folder is evidence of outcome, and the panel claims to show the folder.

Rules that follow from it:

- **one named method per message**; `ipcRenderer` is never exposed, and no bridge function takes a channel name from its caller. The bridge surface must be enumerable by reading the file;
- `invoke / handle` for request→reply, `send / on` only for main pushing state; every subscription hands back an **unsubscribe** or a remounting component leaks a listener;
- **every** inbound payload is `parse d` in main before it reaches a use case — shape validated, sizes bounded, rejected rather than coerced;
- errors cross as **typed data**, not thrown `Error`s, so the UI can tell "no microphone" from "agent timed out" from "model file missing":

```
export type TurnFailure =
  | { kind: 'mic-denied';    denial: 'retryable' | 'system-settings' }
  | { kind: 'no-microphone' }
  | { kind: 'setup';        what: 'agent-cli' | 'agent-auth' | 'whisper' | 'model'; hint: string }
  | { kind: 'transcribe-failed'; stderr: string }
  | { kind: 'agent-failed';  stderr: string }
  | { kind: 'timeout';      afterMs: number }
  | { kind: 'empty-speech' }
```

8. Security switches, written down on purpose

```
new BrowserWindow({
  webPreferences: {
    preload: join(__dirname, '../preload/index.js'),
    contextIsolation: true,    // default since Electron 12 – asserted anyway
    sandbox: true,            // default since Electron 20 – asserted anyway
    nodeIntegration: false,    // default – asserted anyway
  },
})
```

They are the current defaults. Writing them down regardless costs three lines and is the difference between an XSS bug and remote code execution on the user's machine — a default nobody chose is a default anyone can change without a review noticing.

Also set deliberately: `will-navigate` and `setWindowOpenHandler` **deny by default**; a CSP with no `unsafe-eval` (in a desktop app that is a shell); a single-instance lock, because two copies writing the same `notes/` is a bug that will not reproduce.

9. Version marker

One SemVer number, in one place: the `version` field of `package.json`. It is read at build time and rendered in the window during development as `v<version> · dev`.

Bump policy: one bump per merge, weighted by the size of the change — patch for a small one, minor for a backwards-compatible batch, major for a breaking one — written as the last commit on the branch, after sign-off and immediately before the merge. That is the organisation's standard cadence, and this project follows it.

This paragraph used to say something else, and the record of that is worth more than a tidy rewrite. It said *every code change bumps the patch level*, on the argument that the badge was a live signal during manual testing and so had to move whenever the code did. That policy was never followed: the first batch ran to twelve commits and moved the version exactly once, `0.1.0` → `0.2.0`, in a commit whose own message reads *"after sign-off, as the last commit before the merge"* — the standard cadence, to the letter. A rule that the work walked past twelve times in a row is not a rule; it is a paragraph.

And the need behind it was real but misfiled. *"Is the window in front of me the build I just changed?"* is a question about a **commit**, not a release, and the two want to move at different rates. Overloading one canonical SemVer number with both jobs makes it useless for either. If the signal is wanted back, it is a short SHA beside the badge — cheap, exact, and it never has to be decided in a meeting.

10. Localisation

Exactly one language ships: English. Not "English first" — English only. What is built now is the *seam* that makes a second language cheap, and nothing beyond it.

```
src/renderer/i18n/en.json      { "talk.hold": "Hold to talk", "err.mic.title": "...", ... }
src/renderer/i18n/index.ts    t(key: MessageKey): string  — one dictionary, one lookup
```

Built now

- every user-visible string lives in `en.json` and reaches the screen through `t()`;
- `MessageKey` is derived from the keys of `en.json`, so a typo in a key is a compile error and a string added to the UI without a dictionary entry does not build.

Deliberately not built

- no second locale file — no `pl.json`, not even an empty one;
- no language switcher, no locale detection, no `Intl` fallback chain;
- no `i18n` library, no plural rules, no gendered forms, no interpolation. The moment a string genuinely needs a placeholder, `t()` grows one parameter — that is the whole migration.

The reason for building the seam and none of the machinery: the expensive half of localisation is not translating, it is **finding every hard-coded string afterwards**. Doing it from the first commit costs nothing; retrofitting it costs a pass over the entire UI. Adding language two is then one JSON file, one line in the composition of `t()`, and a decision about where the switch lives — recorded as future work, not as a stub sitting in the repository waiting for it.

11. Tech stack, and why

Every version is the current stable at the time of writing, and each one is a choice.

what	version	why this and not the alternative
Electron	44.x	the requirement is a desktop app driving local binaries and the file system; Electron is the shortest path from a web UI to that, and the version line carries <code>windowStatePersistence</code> and the reworked clipboard
TypeScript	7.x	<code>strict</code> . Types at the seams are the review this project cannot afford to run by hand
electron-vite	5.x	gives main / preload / renderer as three build outputs with three tsconfigs — the layer split, for free, instead of a hand-rolled build
Vite	7.x	the renderer dev server and bundler <code>electron-vite</code> builds on. Pinned below the current major on purpose: <code>electron-vite@5</code> declares a peer range on Vite 7, and <code>@vitejs/plugin-react@6</code> wants Vite 8 — so the two cannot both be satisfied today. This row said <code>8.x</code> while <code>package.json</code> pinned <code>7.3.6</code> , which is the kind of drift that sends someone to "fix" the toolchain and rediscover a conflict this repo already resolved
React	19.x	the UI is one screen with one state machine; React's cost here is small and the component vocabulary matches the design deliverable
zod	4.x	schema validation at the two seams that need it: IPC payloads and agent stdout. Chosen over hand-written type guards because a guard that drifts from its type compiles fine
Vitest	5.x	runs the domain and adapter tests with no Electron at all; same config shape as the renderer build
Playwright (<code>_electron</code>)	current	one smoke test that launches the real app. §12 explains why exactly one
whisper.cpp	Homebrew <code>whisper-cpp</code>	§6
<code>say</code>	macOS built-in	§6
Node	26.x	the runtime the toolchain is pinned to; <code>engine-strict</code> refuses to install on anything else rather than warning and continuing

Not used, deliberately: no state-management library (one state machine, held in one hook), no CSS framework (the design deliverable is a small custom surface), no i18n library (§10), no `ffmpeg` (§6 — the transcriber decodes WAV itself), no native keyboard hook (§12).

12. Testing tiers, and what is cut

tier	what it covers	cost
domain unit tests	the turn machine, the 250 ms rule, WAV encoding, reply parsing	no Electron, milliseconds
adapter tests	argv construction, exit-code mapping, <code>ENOENT</code> → setup failure, timeout kills	spawns stubs, not real binaries
architecture test	the dependency rule, proven by breaking it	~15 lines
one Playwright <code>_electron</code> smoke test	the app launches, the preload bridge exists, one round trip crosses the real IPC seam	one launch, seconds

The cut, with its price named: there is no broad end-to-end suite, no packaged-artifact test and no visual regression tier.

After this cut, the classes of defect running against **no** check at all are: a failure that only appears in a **packaged** (asar, signed, sandboxed) build, and any regression in the UI's appearance. What is *not* uncovered is "nothing executes in the renderer" — the single Playwright launch exists precisely so that a CSP or preload-wiring defect, which no unit test can see, still has one gate in front of it.

The native global keyboard hook is cut the same way: hold-to-talk works while the window has focus, so **a hold started while another app is focused does nothing**, and nothing detects that regression except using the app. The alternative costs a native module rebuilt per Electron version plus an Accessibility permission the user grants in System Settings — real money against a requirement the brief states as "hold a key **or** button".

13. Packaging

Decided: out of scope for this delivery. The whole promise is `npm run dev` on a clean machine, which is exactly what the README documents and exactly what gets tested. Packaging, signing, notarisation and a CI/CD pipeline are listed as future work at the end of `docs/WORK-BREAKDOWN.md`.

After this cut, the class of defect running against no check is anything that only appears in a **packaged** build — an `asar` path assumption, a spawned binary that is not in the bundle, a missing `Info.plist` key. Nothing in this repository exercises a packaged artifact, so that whole class arrives, if it arrives, on the day someone first runs `electron-builder`.

The microphone usage description (`NSMicrophoneUsageDescription`) is written into the build configuration anyway. It costs one line, and without it a future packaged build fails `getUserMedia` with no prompt and no explanation — which is the least debuggable failure in this entire application.

14. Decisions taken

The questions this plan opened, and their answers:

1. **Default agent CLI** — `claude -p`, tested against `2.1.263`. `codex exec` and `cursor-agent -p` remain a second adapter each, not a change to the use case.
2. **Spoken reply — in scope**, via the macOS `say` binary behind `SpeechSynthesizer` (§6).
3. **Default Whisper model** — `base.en` (~150 MB). It keeps first-run setup to seconds, which is what the "runs on a clean machine" criterion is actually measuring. `large-v3-turbo` is one `VOICEDesk_WHISPER_MODEL` away and the README says so.
4. **Packaging — out of scope** (§13); dev build only, future work recorded.
5. **Agent model** — `haiku`, **pinned by alias** on every call (§5.2). This is a demonstration build: the cheapest current tier is the right default, and an expensive one must not be reachable by accident. The alias form is the decision — it cannot resolve to an Opus-tier model and it survives the retirement of any dated snapshot; today it resolves to `claude-haiku-4-5`. `VOICEDesk_AGENT_MODEL` (default `haiku`) is the explicit opt-in for anything stronger, so a stronger model is chosen, never inherited.
6. **Agent authentication** — the **CLI's own macOS Keychain login** (§5.5). VoiceDeskElectron passes no API key and holds no credential, so a secret is not in its blast radius; a machine that has never signed in is a *setup* failure with an exact fix, caught by a preflight before the spawn rather than reported as an agent failure.

VoiceDeskElectron — work breakdown

The build, decomposed before any of it is written. Each subtask carries a scope, an acceptance criterion **quoted from the client brief** (not "tests are green"), a complexity rating 1–10 that decides how much review it gets, and a parallelism label.

Status values: `PLANNED` · `IN PROGRESS` · `BLOCKED` · `BUILT` (awaiting acceptance) · `DONE` (accepted) .
`BUILT` means the code exists and its own checks pass; it becomes `DONE` only when a reviewer has checked it against the acceptance criterion above it, which for this build is the iteration's pull request. Nothing is reported done until its acceptance criterion has been checked against the brief — and the umbrella is done only when every subtask is, not when most of them are.

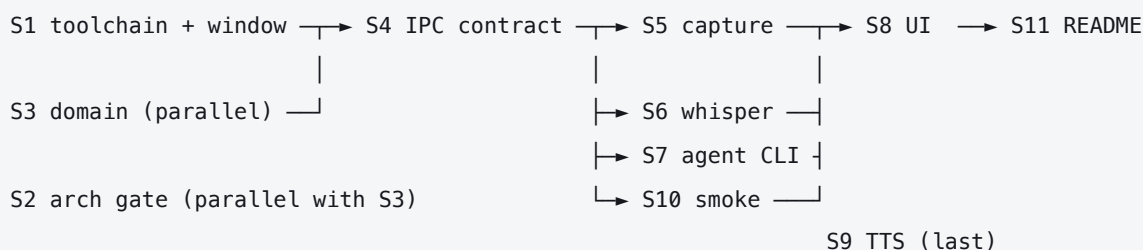
Branching, and a deliberate deviation

The usual rule here is one branch per subtask, squash-merged. **The squash is the part this build drops**, because the brief grades the commit history itself:

"Don't clean up the history. Real commits as you go, not one squash at the end."

A squash merge is exactly the cleanup that sentence forbids. So each iteration is built on its own branch and merged by a pull request **without squashing**: every commit made along the way arrives on `main` as it was written, and the client also gets a reviewable PR per iteration — which committing straight to `main`, the earlier reading of the same rule, would not have given them. The trade is identical either way: `main` carries intermediate states; that is the point.

Dependency shape



Iterations

The build is not one run. What the current iteration contains is stated here, so "not done yet" and "not in scope yet" never look the same from the outside.

iteration	subtasks	what exists at the end of it
1 — merged	S1 → S6	the app opens, records while you hold, shows a level meter, and turns your speech into text on screen. It does not reach the agent yet
2 — current	S7 → S11	the transcript reaches <code>claude -p</code> , edits <code>notes/</code> , and the reply comes back; the approved design is implemented; the reply is spoken; one real launch guards the seam; the README is closed out

Iterations 2 and 3 were merged into one, by the owner's decision, at the start of this run. The table above originally split them. Recording the change rather than quietly rewriting the plan: the trade is a larger PR against a shorter path to a working product, and the owner took it knowingly. Everything else about the branching rule is unchanged — one branch, real commits as they were written, a merge commit and never a squash.

S8 was `BLOCKED` on the design canvas being filled in and approved — the UI is implemented against an approved design, not sketched twice. It is unblocked: the canvas exists at design `0.3.1`, was read directly during this iteration, and S8 was built against it.

Iteration 1 deliberately stopped **before** the agent adapter. Speech-to-text on screen is a complete, demonstrable thing on its own, and it is the half that carries the hardware, the permissions and the process boundaries — the half worth reviewing before anything is layered on top of it.

Subtasks

S1 — toolchain and a window that opens

Scope: stand up Node 26 / TypeScript strict / electron-vite with three build outputs, and a single `BrowserWindow` that opens with the three security switches written explicitly and a version badge reading `package.json`.

- **Acceptance:** *"It runs on a clean machine following your README"* — `npm ci && npm run dev` on a machine that has never seen this repo opens a window showing the version from `package.json` followed by `· dev` — the number itself lives in one place and is not restated here, so a bump cannot make this criterion false.
- **Complexity:** 5 · **Parallelism:** SEQUENTIAL (root) · **Status:** BUILT (awaiting acceptance)

S2 — the architecture gate

Scope: Vitest, plus the ~15-line test that fails when `domain/` imports the platform, proven by breaking it once.

- **Acceptance:** the gate has been *seen* to redden on a deliberate violation, and asserts it scanned a non-empty file list.
- **Complexity:** 3 · **Parallelism:** PARALLEL (with S3) · **Status:** BUILT (awaiting acceptance)

S3 — the domain

Scope: `TurnState`, `TurnFailure`, the three ports, and `runVoiceTurn` as a function taking ports as arguments. Unit tests for the turn machine, including the 250 ms rule and the "key repeat is ignored" rule.

- **Acceptance:** the whole flow *audio* → *transcript* → *reply* is expressible and testable with no Electron, no binaries and no network.
- **Complexity:** 5 · **Parallelism:** PARALLEL (needs nothing from S1) · **Status:** BUILT (awaiting acceptance)

S4 — the IPC contract

Scope: `shared/ipc.ts` (channel names + zod schemas, one declaration), the preload bridge with one named method per message, and main handlers that unwrap → call the use case → wrap. Errors cross as typed data.

- **Acceptance:** the renderer can reach exactly the five declared messages and nothing else; `ipcRenderer` is not exposed and no bridge function takes a caller-supplied channel name.
- **Complexity:** 6 · **Parallelism:** SEQUENTIAL (after S1, S3) · **Status:** BUILT (awaiting acceptance)

S5 — push-to-talk capture

Scope: hold-to-talk on the button and on `Space`; `getUserMedia` on first hold; `AudioContext` at 16 kHz; an `AudioWorklet` producing mono Float32; a live level meter; all four hold-ending events; microphone permission handled as three distinct outcomes.

- **Acceptance:** "Hold a key or button, speak, release → your words appear as text in the window."
- **Complexity:** 7 · **Parallelism:** SEQUENTIAL (after S4) · **Status:** BUILT (awaiting acceptance)

S6 — the transcription adapter

Scope: `WhisperCppTranscriber` over `execFile`, the pure Float32 → 16-bit PCM WAV encoder, binary + model resolution that survives a Finder-launched app's minimal `PATH`, and the `ENOENT` → *setup failure* mapping.

- **Acceptance:** speech becomes text with no API key, no account and no network; a missing binary or model says which one and how to install it.
- **Complexity:** 6 · **Parallelism:** PARALLEL (with S5, S7) · **Status:** BUILT (awaiting acceptance)

S7 — the agent CLI adapter

Scope: `ClaudeCliAgentRunner` and nothing else — `claude -p` **only**. Argv array via `execFile` with no shell, the full flag set from `docs/PLAN.md` §5.2, zod-parsed JSON reply, session continuity carried explicitly, timeout that kills, and the outcome mapping of §5.6. Two choices inside it carry their own reasons: the model is pinned to `--model haiku` — the cheapest current tier, by alias so it can never resolve to an Opus-tier model — with `VOICEDesk_AGENT_MODEL` (default `haiku`) as the deliberate opt-in for anything stronger;

and authentication is the CLI's own macOS Keychain login, so no API key is passed or read and a keychain preflight turns "*never signed in*" into a distinct **setup** failure. No second CLI adapter is written; the port already makes one cheap when it is wanted.

- **Acceptance:** "*add milk to my shopping list*" creates or edits `notes/shopping.md`, and "*what's on my list?*" answers from it — through the CLI, not the raw API, with the agent's blast radius bounded to `notes/`. Two further checks, both answerable without reading the code: the reply envelope's `modelUsage` names a Haiku model, so the cheap tier is proved by the reply rather than assumed; and on a machine that has never signed in, the app says "*run claude in a terminal and log in*" instead of reporting an agent failure.
- **Complexity:** 7 · **Parallelism:** PARALLEL (with S5, S6) · **Status:** BUILT (awaiting acceptance) — the acceptance criterion is executed against the real CLI in `test/claude-cli-agent.test.ts`, including the two further checks: the reply's `modelUsage` names a Haiku model, and a machine with no keychain login gets its own setup failure

S8 — the interface

Scope: implement the approved design — the eight states, the talk control with its meter, the four failure screens, the about/credits sheet, and every string through `t()` against the one locale that ships (English). No second locale, no switcher — see `docs/PLAN.md` §10.

- **Acceptance:** "*UI: generate it with Claude Design ... it looks intentional and is usable*", and each artboard state is recognisable in the running app.
- **Complexity:** 7 · **Parallelism:** SEQUENTIAL (after S5) · **Status:** BUILT (awaiting acceptance) — **unblocked by reading the canvas.** The design project was reachable after all: `Voice Desktop.dc.html` at design `0.3.1` was read directly, which settled every item `docs/design/DESIGN-BRIEF.md` §12 had listed as unverifiable — the fifteen artboard names, the error-code mapping, the copy, the absence of an `empty-speech` board, and the fact that the canvas states no minimum window size.

S9 — spoken reply (*stretch*)

Scope: `MacSaySynthesizer` behind the port, plus the control that triggers it.

- **Acceptance:** "*Stretch, only if you're inside the budget: the reply is spoken back.*"
- **Complexity:** 3 · **Parallelism:** PARALLEL (last) · **Status:** BUILT (awaiting acceptance) — it stayed as cheap as promised: `MacSaySynthesizer` behind the port, one `Speak reply` control on the reply, no new dependency. A voice that will not start withdraws the control rather than failing the turn, which is what `SpeechOutcome`'s third value exists for.

S10 — the one end-to-end

Scope: a single real Electron launch that starts the app shell, asserts the preload bridge exists, and puts one round trip across the real IPC seam.

Built as `test/preload-bridge.test.ts` **plus** `test/fixtures/bridge-probe.cjs`, **not with Playwright.** The launch already existed for S4 and does the three things only a launched app can show: a sandboxed preload has no module resolver, so a preload that typechecks and bundles can still fail at load and leave

`window.voicedesk` undefined; the Content-Security-Policy is a header and so exists only when something serves it; and the seam itself is `renderer → preload → ipcMain.handle → back`. Adding Playwright would have bought a second launcher for a launch this repository already performs.

What that cut stops catching, recorded rather than left implied: nothing drives the UI as a user does. There is no test that presses the talk control, holds `Space`, or reads what the window renders, so a control wired to the wrong handler, a state that renders the wrong pane, or a button that is disabled when it should not be will not redden anything. The seam is gated; the interaction is not.

- **Acceptance:** something in this project executes in the renderer, so a CSP or preload-wiring defect has a gate in front of it.
- **Complexity:** 4 · **Parallelism:** SEQUENTIAL (after S4, S5) · **Status:** BUILT (awaiting acceptance) — **and delivered without Playwright**, which is a cut that names its own cost below.

S11 — the README

Scope: install from zero on a Mac that has nothing; run; what works; what was cut and what that stops catching; how long it actually took; and the full third-party licence list with names and versions, which is also the data behind the about panel.

- **Acceptance:** *"It runs on a clean machine following your README" and "A README that tells the truth"*.
- **Complexity:** 4 · **Parallelism:** SEQUENTIAL (written incrementally from S1 onward, closed last) · **Status:** BUILT (awaiting acceptance)

How each subtask is run

Complexity decides the shape of the loop, not the mood:

rating	loop
< 2	done in the main thread, no ceremony
2–6	develop → test → review, one reviewer
≥ 7	develop → test → review, then a panel of independent critics before acceptance

S5, S7 and S8 are the three that earn the panel. S4 and S6 sit just under it and get a full single review. Nothing is marked done on a green test alone — the gate is the acceptance criterion above it, checked against the brief.

Answered before the build started

1. **Default Whisper model** — `base.en` (~150 MB). Fast first run beats accuracy on a criterion that is literally "it runs on a clean machine following your README". `large-v3-turbo` stays one environment variable away.

2. **Spoken reply (S9)** — in scope, via the macOS `say` binary.
 3. **Packaging** — not in this delivery. `npm run dev` is the whole promise, and the cost of that cut is written into `docs/PLAN.md` §13 rather than left implied.
-

Future work — not in this delivery

Recorded so the absence is a decision rather than an oversight. None of it is started, none of it is promised, and the README does not claim any of it.

#	item	why it is not now
F1	Packaged build — <code>electron-builder</code> producing <code>.app</code> and <code>.dmg</code> , with <code>NSMicrophoneUsageDescription</code> in <code>Info.plist</code>	the delivery is a dev build; nothing in the repo exercises a packaged artifact today
F2	Code signing, notarisation, stapling	needs a paid Apple Developer account and certificates on the build machine — an external dependency, not an engineering decision
F3	CI on GitHub Actions — typecheck, lint, the unit suite, the architecture gate and the real Electron launches on every push, on a macOS runner because <code>package.json</code> declares <code>"os": ["darwin"]</code> and three of the adapters are macOS-only	the gates exist and run locally; wiring them to a runner is a day's work that buys nothing until more than one person commits. GitHub Actions specifically, because it is what the rest of the tooling around this project already speaks
F4	CD — tagged release publishing the signed artifact	strictly after F1–F3; a release pipeline without a signed artifact publishes something nobody can open
F5	Auto-update	only meaningful once F4 exists
F6	Second agent adapter (<code>codex exec</code> , <code>cursor-agent -p</code>)	the port exists from day one, so this is an adapter and a line in the composition root — cheap later, wasted now
F7	Second language	the lookup table and <code>t()</code> ship in this build; a locale is a JSON file and a switch
F8	Global hold-to-talk while another app is focused	needs a native keyboard hook rebuilt per Electron version plus an Accessibility permission — see <code>docs/PLAN.md §12</code>
F9	Running the agent on a larger model tier	not an omission: <code>haiku</code> is the default on purpose because this is a demonstration build and an expensive tier must not be reachable by accident. <code>VOICEDesk_AGENT_MODEL</code> is now a three-value enumeration — <code>haiku</code> , <code>sonnet</code> , <code>opus</code> — so an operator opts in deliberately and by name , and a typo is refused with a setup failure listing the legal values rather than silently substituting the default. Nothing in this delivery raises the tier by itself, and no quality claim is made for one that does; <code>haiku</code> is the only tier this build has been exercised on
F10	Whisper inside the app — the <code>whisper-cli</code> binary, its libraries and a model shipped with the build, so the install is one step instead of four	the delivery is a dev build, and vendoring a binary is a packaging decision that belongs after F1. Measured on the development machine: <code>whisper-cli</code> is 643 KB and its libraries about 1.1 MB, and the <code>base.en</code> model is 141 MB — so ~143 MB, effectively all of it the model. The size is therefore a model choice, not an engine one, and <code>tiny.en</code> would trade accuracy for about 100 MB of it. The size increase is accepted on purpose: a user who has to install four things has four chances to give up before the app does anything
F11	Choose the notes folder from the app — a directory picker writing what <code>VOICEDesk_NOTES_DIR</code> sets today	an environment variable is enough for one operator on one machine and nothing else; it needs a settings surface, which this build does not have

#	item	why it is not now
F12	A notes list you can use — click a row to open the file, delete a row	the panel was specified as a <i>readout</i> of what the agent touched (docs/design/DESIGN-BRIEF.md §5), so nothing in it is interactive. Making a row clickable is small; making one deletable needs a confirmation, an undo or both, because it destroys the user's own writing
F13	One conversation per note — the agent dialogue attached to the note it is about, and itself deletable, instead of one session for the whole app	sessionId is carried as a single value in useTurn , which is the right shape for one conversation and the wrong shape for many. This is a data-model change, not a UI one
F14	The agent may delete a Markdown file	NotesFolder exposes reading and listing; deletion is the one file operation deliberately absent, because an agent that can delete on a misheard sentence is a different risk class from one that can only add. It needs the confirmation story from F12 first
F15	Rename a note	the name comes from whatever the agent chose on the first sentence, and there is no way to change it afterwards
F16	Sign in as any account , instead of inheriting whichever account Claude Code is logged into on this machine — and, with it, a deliberate way to clear the conversation context	the app authenticates by <i>not</i> authenticating: it checks that Claude Code's own keychain entry exists and never reads it (README.md → Requirements). That is the cheapest correct thing for a demonstration build and the wrong thing for a product, because the user cannot choose the account and cannot start over
F17	A UI happy-path gate that runs after the merge — drives the window through one whole turn and reports, rather than standing in front of the merge	this is the coverage the Playwright cut gave up, named in the cut table above: nothing drives the UI as a user does. Putting it <i>after</i> the domain gate rather than in front of it is the point — a flaky browser check that can block a merge gets disabled within a month, and one that only reports does not
F18	The microphone permission belongs to "Electron" — System Settings shows Electron's name and icon, not the app's, and every Electron app on the machine shares that one switch	an unpackaged development build has no bundle identifier of its own, so it <i>is</i> Electron as far as macOS is concerned. Fixed by F1 and F2 and by nothing short of them
F19	**A missing microphone permission can surface as "Setup is incomplete"*** with a filesystem path in the body	reported from use and not reproduced here, because this machine has the permission granted. setup is the board for things that are not installed, and its body is the failing adapter's hint verbatim — two of which quote an absolute path. A permission problem must not be routed through it, and an absolute path does not belong mid-sentence in front of a user. It survived because the microphone path is the one the README's " <i>Needs a human</i> " note says has no automated gate